MINI PROJECT - ELECTRICITY CONSUMPTION FORECASTING USING MACHINE LEARNING

AIM:

To forecast future electricity consumption using historical time series data by implementing a **Linear Regression** model with weather, calendar and lag features, to evaluate its performance using error metrics, and to deploy it as a login-protected Flask web application.

ALGORITHM:

1. Data Collection: Obtain historical daily electricity consumption data (kWh) with temperature (e.g., UCI Household Power Consumption or a Kaggle energy dataset).
2. Data Preprocessing: Handle missing values using interpolation.
3. Convert the Date column to datetime format.
4. Sort the data chronologically.
5. Feature Engineering: Create calendar features (DayOfWeek, Month) and lag features (Lag_1 = previous day, Lag_7 = same day last week).
6. Feature Selection: Use Temperature, DayOfWeek, Month, Lag_1 and Lag_7 as inputs and Consumption as the target.
7. Train-Test Split: Divide the data chronologically into training and testing sets (80%-20%) without shuffling.
8. Model Training: Fit a Linear Regression model using the training data.
9. Prediction: Use the trained model to predict electricity consumption on the test set.
10. Evaluation: Assess the model using Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE) and R^2 score.
11. Visualization: Plot actual vs predicted consumption to observe forecasting accuracy.
12. Deployment: Save the model with joblib and load it in a Flask application with Register, Login and Dashboard pages.
13. Prediction on the Web: Accept user inputs through a form and display the predicted consumption in kWh.

CODE:

train_model.py

```
import os
import json
import joblib
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

CSV_PATH = "electricity_data.csv"      # columns: Date, Temperature, Consumption

# ---------- 1. Data Collection ----------
def load_data():
```

```
    if os.path.exists(CSV_PATH):
        return pd.read_csv(CSV_PATH)
    # Simulated daily data (same schema as a real dataset) used when no CSV is present
    rng = np.random.default_rng(42)
    dates = pd.date_range("2022-01-01", "2024-12-31", freq="D")
    doy = dates.dayofyear.values
    temp = 28 + 6 * np.sin(2 * np.pi * (doy - 100) / 365) + rng.normal(0, 1.5,
        len(dates))
    noise = np.zeros(len(dates))
    for t in range(1, len(dates)):
        noise[t] = 0.6 * noise[t - 1] + rng.normal(0, 6)
    weekend = (dates.dayofweek >= 5).astype(int)
    trend = np.linspace(0, 15, len(dates))
    kwh = 220 + 7 * (temp - 28) + 18 * weekend + trend + noise
    df = pd.DataFrame({"Date": dates, "Temperature": temp.round(1), "Consumption":
        kwh.round(2)})
    df.to_csv(CSV_PATH, index=False)
    return df

df = load_data()

# ---------- 2. Data Preprocessing ----------
df["Date"] = pd.to_datetime(df["Date"])
df = df.sort_values("Date").reset_index(drop=True)
df = df.set_index("Date").interpolate().reset_index()      # handle missing values

# ---------- 3. Feature Engineering ----------
df["DayOfWeek"] = df["Date"].dt.dayofweek
df["Month"] = df["Date"].dt.month
df["Lag_1"] = df["Consumption"].shift(1)                    # yesterday
df["Lag_7"] = df["Consumption"].shift(7)                    # same day last week
df = df.dropna().reset_index(drop=True)

FEATURES = ["Temperature", "DayOfWeek", "Month", "Lag_1", "Lag_7"]
X, y = df[FEATURES], df["Consumption"]

# ---------- 4. Chronological Train-Test Split (80-20) ----------
split = int(len(df) * 0.8)
X_train, X_test = X.iloc[:split], X.iloc[split:]
y_train, y_test = y.iloc[:split], y.iloc[split:]

# ---------- 5. Model Training ----------
model = LinearRegression()
model.fit(X_train, y_train)

# ---------- 6. Prediction and Evaluation ----------
pred = model.predict(X_test)
mse = mean_squared_error(y_test, pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, pred)
r2 = r2_score(y_test, pred)

print("Total records      :", len(df))
print("Training records   :", len(X_train))
print("Testing records    :", len(X_test))
print("Intercept          :", round(model.intercept_, 4))
for name, coef in zip(FEATURES, model.coef_):
    print(f"Coefficient {name:<12}: {coef:.4f}")
print(f"MAE                : {mae:.4f} kWh")
print(f"MSE                : {mse:.4f}")
print(f"RMSE               : {rmse:.4f} kWh")
print(f"R2 Score           : {r2:.4f}")

# ---------- 7. Visualization ----------
plt.figure(figsize=(10, 4.5))
plt.plot(df["Date"].iloc[split:], y_test.values, label="Actual", color="#1f77b4")
plt.plot(df["Date"].iloc[split:], pred, label="Predicted", color="#ff7f0e",
    linestyle="--")
plt.title("Electricity Consumption: Actual vs Predicted (Test Set)")
plt.xlabel("Date")
plt.ylabel("Consumption (kWh)")
```

```
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.savefig("actual_vs_predicted.png", dpi=200)

# ---------- 8. Save model for the web app ----------
joblib.dump(model, "model.pkl")
json.dump({"mae": mae, "mse": mse, "rmse": rmse, "r2": r2, "train": len(X_train),
    "test": len(X_test)},
          open("metrics.json", "w"))
```

Index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Electricity Consumption Predictor</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      rel="stylesheet">
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
<div class="container" style="max-width:520px; margin-top:40px">
  <div class="card-form">
    <h2 class="brand">Electricity Consumption Predictor</h2>
    <h5>Welcome, {{ username }}!</h5>
    <form action="{{ url_for('dashboard') }}" method="post" class="mt-3">
      <input type="number" step="0.1" name="Temperature" value="{{
          request.form.get('Temperature', '') }}" placeholder="Temperature (°C)"
             class="form-control mb-3" required>
      <select name="DayOfWeek" class="form-select mb-3" required>
        <option value="">Day of Week</option>
        {% for i, d in [(0,'Monday'),(1,'Tuesday'),(2,'Wednesday'),(3,'Thursday'),
                        (4,'Friday'),(5,'Saturday'),(6,'Sunday')] %}
        <option value="{{ i }}" {{ 'selected' if request.form.get('DayOfWeek') ==
            i|string }}>{{ d }}</option>
        {% endfor %}
      </select>
      <input type="number" min="1" max="12" name="Month" value="{{
          request.form.get('Month', '') }}" placeholder="Month (1-12)"
             class="form-control mb-3" required>
      <input type="number" step="0.01" name="Lag_1" value="{{ request.form.get('Lag_1',
          '') }}" placeholder="Yesterday's consumption (kWh)"
             class="form-control mb-3" required>
      <input type="number" step="0.01" name="Lag_7" value="{{ request.form.get('Lag_7',
          '') }}" placeholder="Consumption 7 days ago (kWh)"
             class="form-control mb-3" required>
      <button type="submit" class="btn btn-power w-100">Predict</button>
    </form>
    {% if prediction_text %}
      <div class="result-box">{{ prediction_text }}</div>
    {% endif %}
    <a href="{{ url_for('logout') }}" class="d-block text-center mt-3">Logout</a>
  </div>
</div>
<div class="footer-dark">Electricity Consumption Predictor &copy; 2025</div>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.j
    s"></script>
</body>
</html>
```

Home.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Electricity Consumption Forecasting</title>
```

```
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      rel="stylesheet">
  <link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-
      awesome/6.4.2/css/all.min.css">
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
<div class="container">
  <header class="d-flex flex-wrap justify-content-between align-items-center py-3 mb-4
      border-bottom">
    <span class="fs-3 brand"><i class="fas fa-bolt"></i> Electricity Forecasting</span>
    <ul class="nav nav-pills">
      <li class="nav-item"><a href="{{ url_for('home') }}" class="nav-link active"
          style="background:#d84315;color:#fff;border-radius:10px">Home</a></li>
      <li class="nav-item"><a href="#features" class="nav-link">Features</a></li>
      <li class="nav-item"><a href="#about" class="nav-link">About</a></li>
      <li class="nav-item"><a href="{{ url_for('register') }}" class="nav-
          link">Register</a></li>
      <li class="nav-item"><a href="{{ url_for('login') }}" class="nav-
          link">Login</a></li>
      <li class="nav-item"><a href="{{ url_for('login') }}" class="btn btn-power
          ms-2">Get Started</a></li>
    </ul>
  </header>
</div>

<div class="container py-4">
  <div class="row align-items-center g-5">
    <div class="col-lg-6">
      <h1 class="hero-title">Electricity Consumption Forecasting</h1>
      <p class="lead">Predict daily electricity usage in kWh using a machine learning
          model trained on
        past consumption, temperature and calendar patterns.</p>
      <a href="{{ url_for('login') }}" class="btn btn-lg btn-power px-4">Get Started</a>
    </div>
    <div class="col-lg-6 text-center"><i class="fas fa-plug-circle-bolt" style="font-
        size:12rem;color:#ff9800;filter:drop-shadow(0 10px 18px
        rgba(216,67,21,.35))"></i></div>
  </div>
</div>

<div class="container py-5" id="features">
  <h2 class="text-center brand mb-4">Features</h2>
  <div class="row g-4 text-center feature">
    <div class="col-md-4"><div class="feature-card"><i class="fas fa-chart-
        line"></i><h4>Accurate Forecasts</h4>
      <p>Linear Regression with lag features captures daily and weekly demand
          patterns.</p></div></div>
    <div class="col-md-4"><div class="feature-card"><i class="fas fa-temperature-
        half"></i><h4>Weather Aware</h4>
      <p>Temperature is used as a key input because cooling load drives
          consumption.</p></div></div>
    <div class="col-md-4"><div class="feature-card"><i class="fas fa-user-
        check"></i><h4>Secure and Simple</h4>
      <p>Login-protected dashboard with a simple form for instant
          predictions.</p></div></div>
  </div>
</div>

<div class="container py-5" id="about">
  <h2 class="text-center brand mb-4">About Us</h2>
  <p class="text-center mx-auto" style="max-width:720px">Electricity Consumption
      Forecasting is a tool that
    uses historical usage data to estimate future demand. Accurate forecasts help
        households and utilities
    plan load, reduce wastage and control costs.</p>
</div>

<div class="footer-dark">&copy; 2025 Electricity Consumption Forecasting</div>
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.j
    s"></script>
```

```
</body>
</html>
```

Login.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Login</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      rel="stylesheet">
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body class="d-flex align-items-center justify-content-center" style="min-height:100vh">
<div class="card-form" style="width:420px">
  <h2 class="text-center brand mb-4">Login</h2>
  {% with messages = get_flashed_messages(with_categories=true) %}
    {% for category, message in messages %}
      <div class="alert alert-{{ 'danger' if category == 'error' else 'success' }}">{{
          message }}</div>
    {% endfor %}
  {% endwith %}
  <form action="{{ url_for('login') }}" method="post">
    <div class="mb-3">
      <label for="username" class="form-label">Username</label>
      <input type="text" class="form-control" name="username" id="username"
             placeholder="Enter your username" required>
    </div>
    <div class="mb-3">
      <label for="password" class="form-label">Password</label>
      <input type="password" class="form-control" name="password" id="password"
             placeholder="Enter your password" required>
    </div>
    <div class="d-grid"><button type="submit" class="btn btn-power">Login</button></div>
    <p class="text-center mt-3">Don't have an account? <a href="{{ url_for('register')
        }}">Register</a></p>
  </form>
</div>
</body>
</html>
```

Register.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Register</title>
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
      rel="stylesheet">
  <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body class="d-flex align-items-center justify-content-center" style="min-height:100vh">
<div class="card-form" style="width:420px">
  <h2 class="text-center brand mb-4">Register</h2>
  {% with messages = get_flashed_messages(with_categories=true) %}
    {% for category, message in messages %}
      <div class="alert alert-{{ 'danger' if category == 'error' else 'success' }}">{{
          message }}</div>
    {% endfor %}
  {% endwith %}
  <form action="{{ url_for('register') }}" method="post">
    <div class="mb-3">
      <label for="name" class="form-label">Name</label>
      <input type="text" class="form-control" name="username" id="name"
          placeholder="Enter your name" required>
    </div>
    <div class="mb-3">
```

```
      <label for="gender" class="form-label">Gender</label>
      <select class="form-select" name="gender" id="gender" required>
        <option value="">Select Gender</option>
        <option value="Male">Male</option>
        <option value="Female">Female</option>
        <option value="Other">Other</option>
      </select>
    </div>
    <div class="mb-3">
      <label for="age" class="form-label">Age</label>
      <input type="number" class="form-control" name="age" id="age" placeholder="Enter
          your age" required>
    </div>
    <div class="mb-3">
      <label for="password" class="form-label">Password</label>
      <input type="password" class="form-control" name="password" id="password"
             placeholder="Enter password" required>
    </div>
    <div class="d-grid"><button type="submit" class="btn btn-
        power">Register</button></div>
    <p class="text-center mt-3">Already have an account? <a href="{{ url_for('login')
        }}">Sign In</a></p>
  </form>
</div>
</body>
</html>
```

Style.css

```
body { font-family: 'Segoe UI', Arial, sans-serif; color: #1f2937;
       background: linear-gradient(135deg, #fff7e0 0%, #ffe9c7 100%); min-height: 100vh;
             }
.brand { color: #d84315; font-weight: 700; }
.btn-power { background: linear-gradient(90deg, #ff9800, #f4511e); color: #fff; font-
    weight: 600;
             border: none; border-radius: 10px; padding: 10px 20px;
             box-shadow: 0 4px 12px rgba(244, 81, 30, 0.35); }
.btn-power:hover { color: #fff; filter: brightness(1.08); }
.card-form { background: #ffffff; border-radius: 18px; padding: 36px;
             box-shadow: 0 12px 35px rgba(216, 67, 21, 0.18); border-top: 6px solid
                 #ff9800; }
.form-control, .form-select { border-radius: 10px; padding: 10px 14px; border: 1px solid
     #e5e7eb; }
.form-control:focus, .form-select:focus { border-color: #ff9800; box-shadow: 0 0 0 3px
    rgba(255, 152, 0, 0.25); }
.nav-link { color: #d84315; font-weight: 500; }
.hero-title { font-size: 2.8rem; font-weight: 800; line-height: 1.15;
              background: linear-gradient(90deg, #d84315, #ff9800); -webkit-background-
                  clip: text;
              background-clip: text; color: transparent; }
.feature-card { background: #fff; border-radius: 16px; padding: 28px 20px; height: 100%;
                box-shadow: 0 8px 24px rgba(0, 0, 0, 0.08); }
.feature i { font-size: 2.2rem; color: #f4511e; margin-bottom: 10px; }
.result-box { background: #e8f5e9; border-left: 5px solid #2e7d32; border-radius: 8px;
              padding: 14px; margin-top: 18px; font-weight: 600; color: #1b5e20; text-
                  align: center; }
.footer-dark { background: linear-gradient(90deg, #d84315, #ff9800); color: #fff;
               text-align: center; padding: 16px; margin-top: 40px; }
```

App.py

```
import joblib
import pandas as pd
from flask import Flask, render_template, request, redirect, url_for, flash
from flask_sqlalchemy import SQLAlchemy
from flask_login import (LoginManager, UserMixin, login_user,
                         login_required, logout_user, current_user)
from werkzeug.security import generate_password_hash, check_password_hash

app = Flask(__name__)
app.config['SECRET_KEY'] = 'change-this-secret-key'
```

```
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///site.db'
db = SQLAlchemy(app)

login_manager = LoginManager(app)
login_manager.login_view = 'login'

model = joblib.load('model.pkl')          # trained Linear Regression model
FEATURES = ['Temperature', 'DayOfWeek', 'Month', 'Lag_1', 'Lag_7']

# --- Models ---
class User(UserMixin, db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(150), unique=True, nullable=False)
    gender = db.Column(db.String(20))
    age = db.Column(db.Integer)
    password = db.Column(db.String(200), nullable=False)

@login_manager.user_loader
def load_user(user_id):
    return db.session.get(User, int(user_id))

# --- Routes ---
@app.route('/')
def home():
    return render_template('Home.html')

@app.route('/register', methods=['GET', 'POST'])
def register():
    if request.method == 'POST':
        username = request.form['username']
        if User.query.filter_by(username=username).first():
            flash('User already exists', 'error')
            return redirect(url_for('register'))
        user = User(username=username,
                    gender=request.form['gender'],
                    age=int(request.form['age']),
                    password=generate_password_hash(request.form['password']))
        db.session.add(user)
        db.session.commit()
        flash('Registered successfully', 'success')
        return redirect(url_for('login'))
    return render_template('Register.html')

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        user = User.query.filter_by(username=request.form['username']).first()
        if user and check_password_hash(user.password, request.form['password']):
            login_user(user)
            return redirect(url_for('dashboard'))
        flash('Invalid credentials', 'error')
    return render_template('Login.html')

@app.route('/dashboard', methods=['GET', 'POST'])
@login_required
def dashboard():
    prediction_text = ""
    if request.method == 'POST':
        values = [[float(request.form['Temperature']),
                   int(request.form['DayOfWeek']),
                   int(request.form['Month']),
                   float(request.form['Lag_1']),
                   float(request.form['Lag_7'])]]
        row = pd.DataFrame(values, columns=FEATURES)
        predicted = model.predict(row)[0]
        prediction_text = f"Predicted Electricity Consumption: {predicted:.2f} kWh"
    return render_template('Index.html', username=current_user.username,
                           prediction_text=prediction_text)

@app.route('/logout')
@login_required
```

```
def logout():
    logout_user()
    return redirect(url_for('login'))

with app.app_context():
    db.create_all()

if __name__ == '__main__':
    app.run(debug=True)
```

OUTPUT:

Model training and evaluation (console output)

```
Total records      : 1089
Training records   : 871
Testing records    : 218
Intercept          : 15.9042
Coefficient Temperature : 5.5206
Coefficient DayOfWeek   : 2.9444
Coefficient Month       : 0.1443
Coefficient Lag_1       : 0.1554
Coefficient Lag_7       : 0.0604
MAE                : 9.0233 kWh
MSE                : 118.4671
RMSE               : 10.8843 kWh
R2 Score           : 0.8837
```

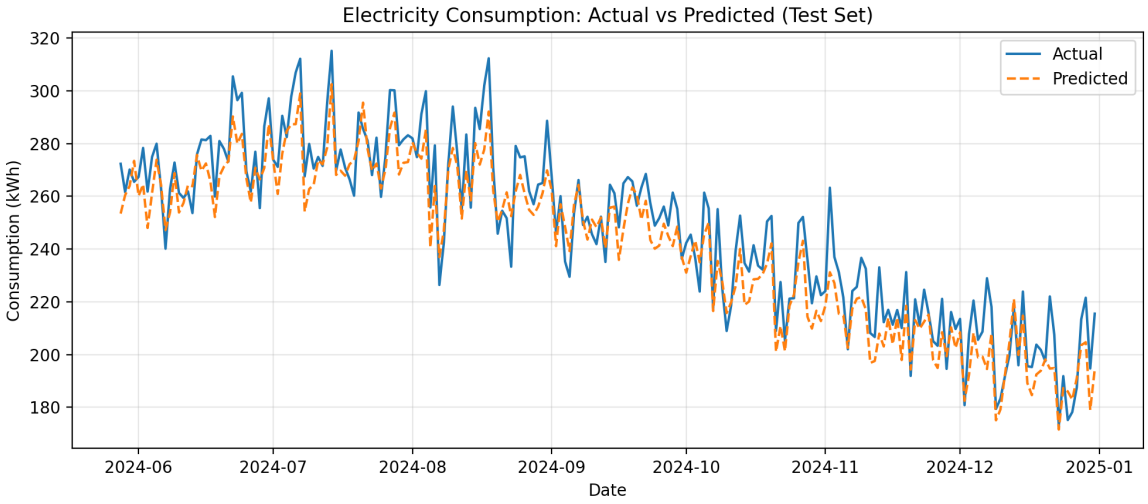


Fig. 1: Actual vs predicted electricity consumption on the test set

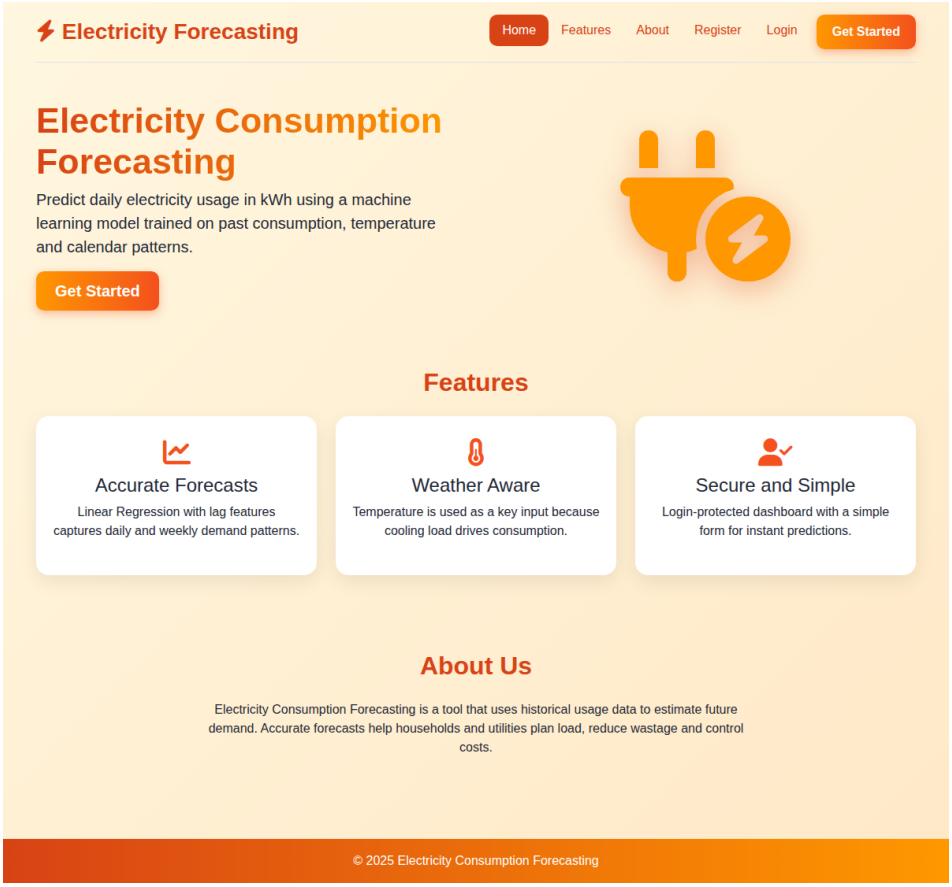


Fig. 2: Home page

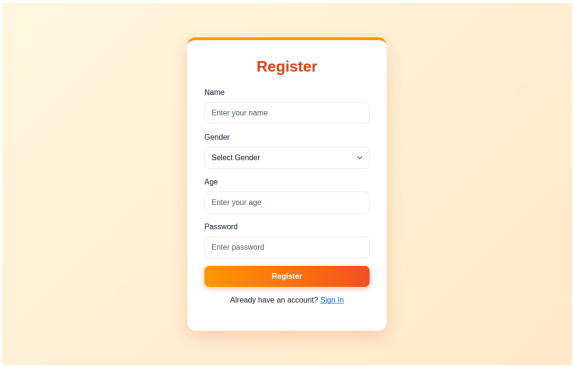


Fig. 3: Register page

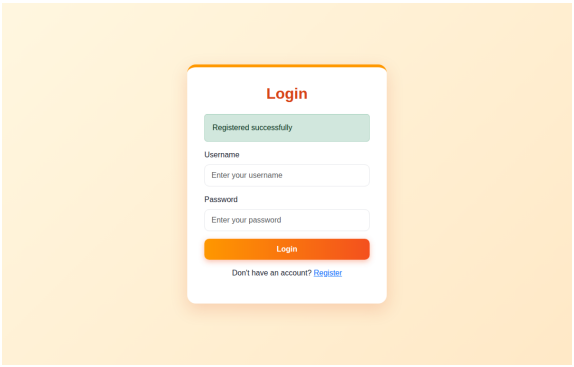


Fig. 4: Login page

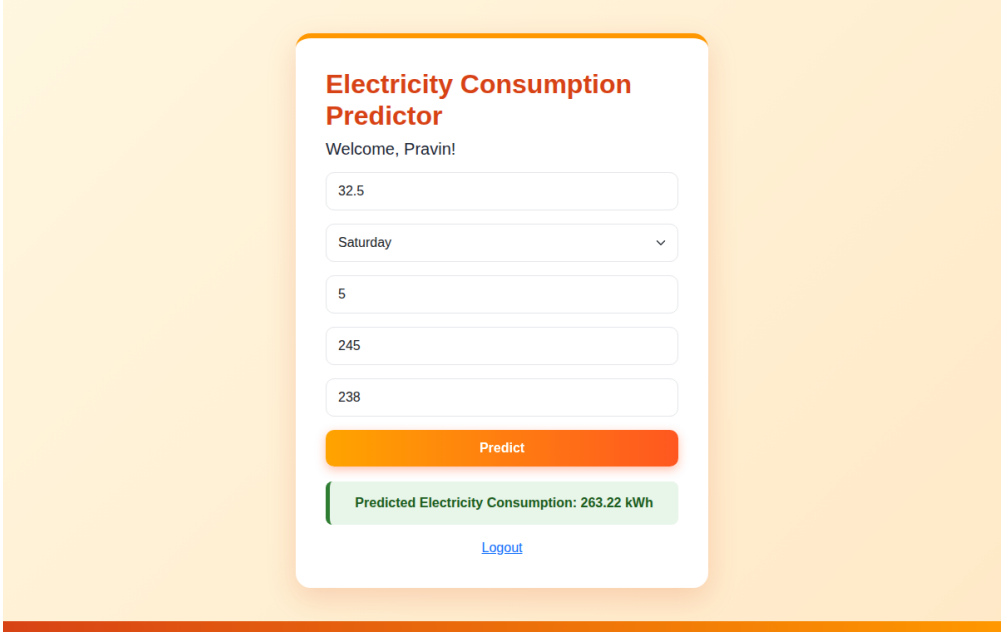


Fig. 5: Dashboard with prediction result

RESULT:

The Linear Regression model was successfully trained on 871 records and tested on 218 records to forecast daily electricity consumption. It achieved an R^2 score of 0.8837, an RMSE of 10.88 kWh and an MAE of 9.02 kWh on the test set. The predicted values closely followed the actual consumption, showing that the model effectively captured the temperature-driven and weekly demand pattern. The trained model was deployed in a Flask web application that returns the predicted consumption in kWh for user-entered inputs.